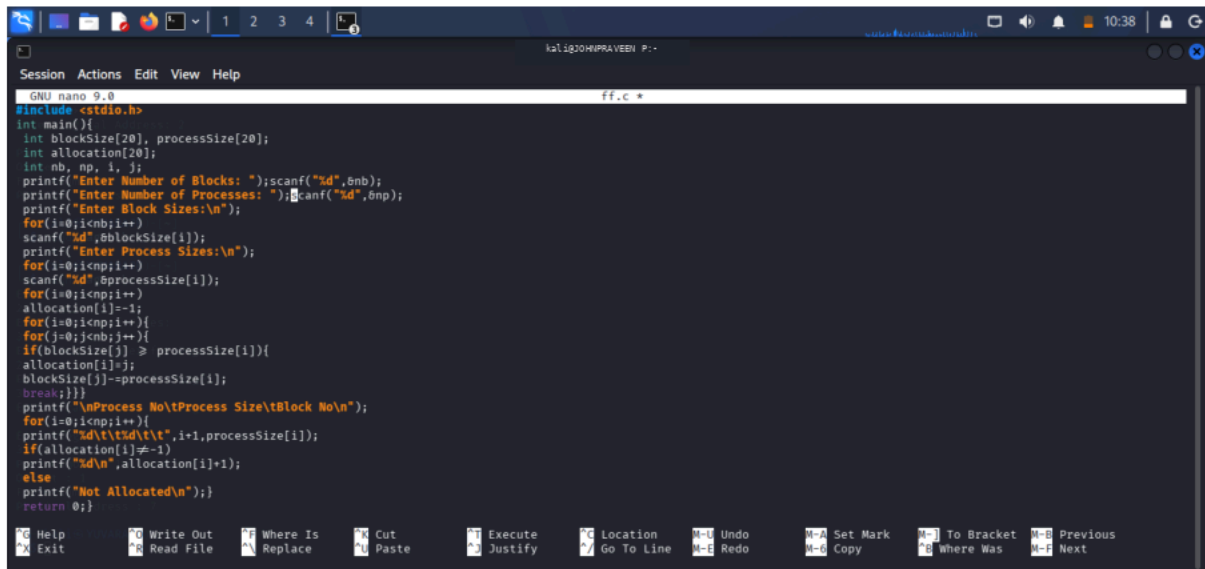
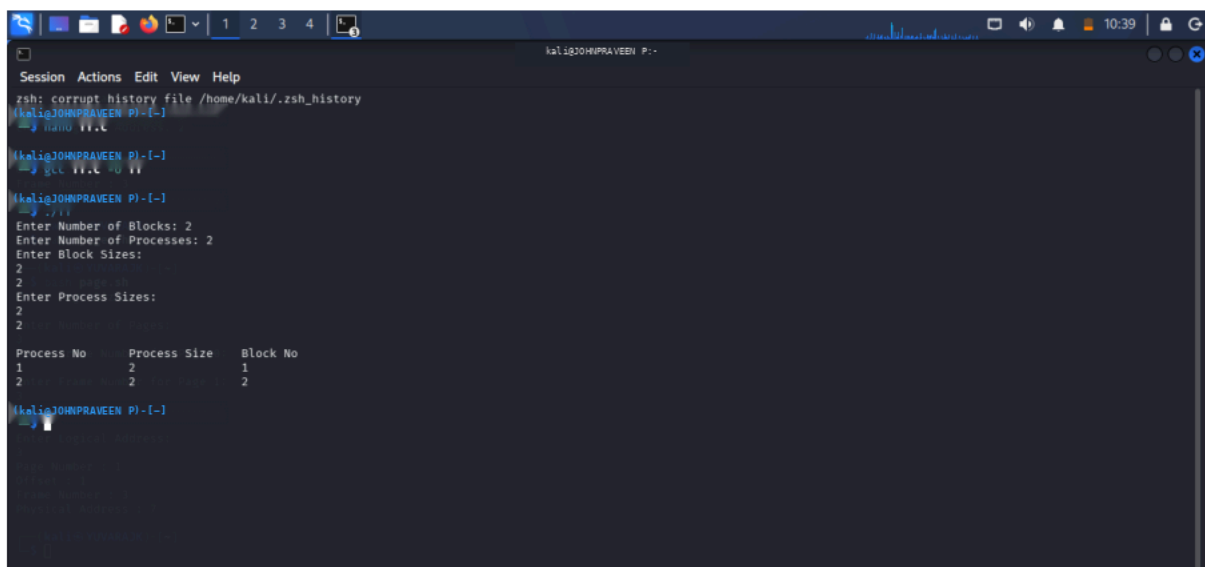


C PROGRAMMING :FIRST FIT



```
GNU nano 9.0 ff.c
#include <stdio.h>
int main(){
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb, np, i, j;
    printf("Enter Number of Blocks: ");scanf("%d",&nb);
    printf("Enter Number of Processes: ");scanf("%d",&np);
    printf("Enter Block Sizes:\n");
    for(i=0;i<nb;i++){
        scanf("%d",&blockSize[i]);
        printf("Enter Process Sizes:\n");
        for(i=0;i<np;i++){
            scanf("%d",&processSize[i]);
            for(j=0;j<nb;j++){
                allocation[i]=-1;
                if(blockSize[j] >= processSize[i]){
                    allocation[i]=j;
                    blockSize[j]-=processSize[i];
                    break;
                }
            }
            printf("\nProcess No\tProcess Size\tBlock No\n");
            for(i=0;i<np;i++){
                printf("%d\t%d\t%d",i+1,processSize[i],
                    if(allocation[i]!=-1)
                        printf("%d\n",allocation[i]+1);
                    else
                        printf("Not Allocated\n");
            }
            return 0;
        }
    }
}
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHNPRABEEN P)-[~]
$ nano ff.c
(kali@JOHNPRABEEN P)-[~]
$ gcc ff.c -o ff
(kali@JOHNPRABEEN P)-[~]
$ ./ff
Enter Number of Blocks: 2
Enter Number of Processes: 2
Enter Block Sizes:
2
2
Enter Process Sizes:
2
2
Process No      Process Size    Block No
1                2                1
2                2                2
(kali@JOHNPRABEEN P)-[~]
$
```

SHELL PROGRAMMING : FIRST FIT

The screenshot shows a Kali Linux terminal window with a nano editor open. The terminal title is 'kali@kali:~\$'. The nano editor is editing a file named 'ff.sh'. The content of the file is as follows:

```
GNU nano 9.0
ff.sh
echo "First Fit Memory Allocation Demonstration"
62
blocks=(100 500 200 300 600)
processes=(212 417 112 426)
echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"
echo "Allocation Performed Using First Fit"
```

The bottom status bar of the nano editor displays various keyboard shortcuts:

- Ctrl+H Help
- Ctrl+W Write Out
- Ctrl+N Where Is
- Ctrl+U Cut
- Ctrl+E Execute
- Ctrl+J Justify
- Ctrl+_ Location
- Ctrl+L Go To Line
- Ctrl+M Undo
- Ctrl+R Redo
- Ctrl+M Set Mark
- Ctrl+Y Copy
- Ctrl+T To Bracket
- Ctrl+W Where Was
- Ctrl+P Previous
- Ctrl+N Next

OUTPUT :

```

kali@kali:~$ nano ff.c
Enter Number of Blocks: 2
Enter Number of Processes: 2
Enter Block Sizes:
2
2
Enter Process Sizes:
2
2

Process No      Process Size      Block No
1                2                  1
2                2                  2

kali@kali:~$ gcc ff.c -o ff
kali@kali:~$ ./ff
Enter Number of Blocks: 2
Enter Number of Processes: 2
Enter Block Sizes:
2
2
Enter Process Sizes:
2
2

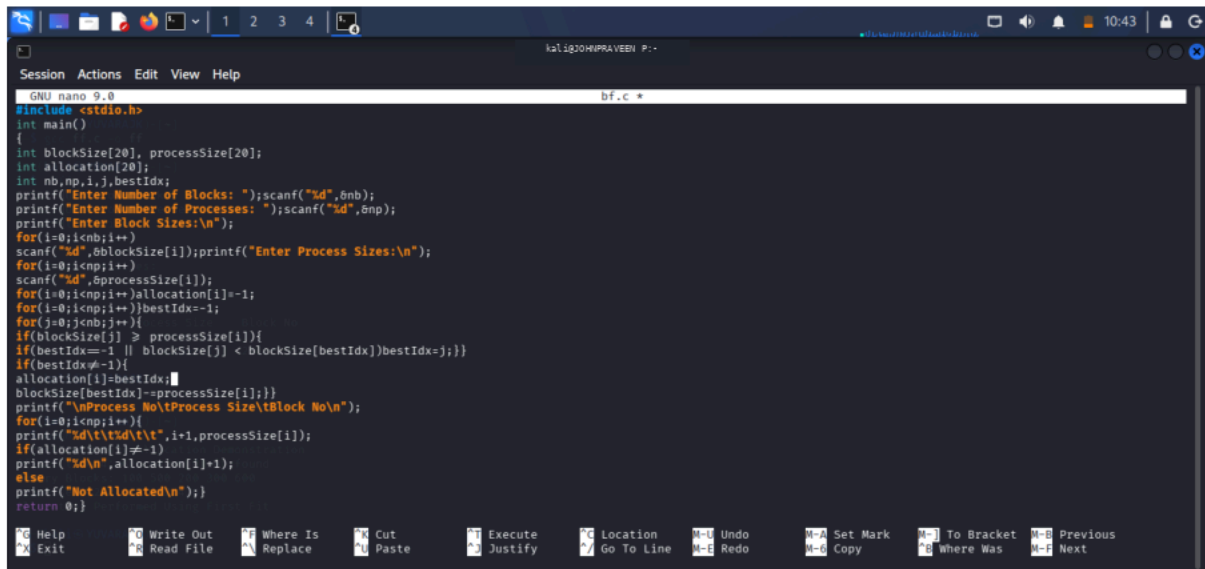
Process No      Process Size      Block No
1                2                  1
2                2                  2

kali@kali:~$ ./ff.sh
First Fit Memory Allocation Demonstration
ff.sh: line 4: 62: command not found
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426
Allocation Performed Using First Fit

kali@kali:~$

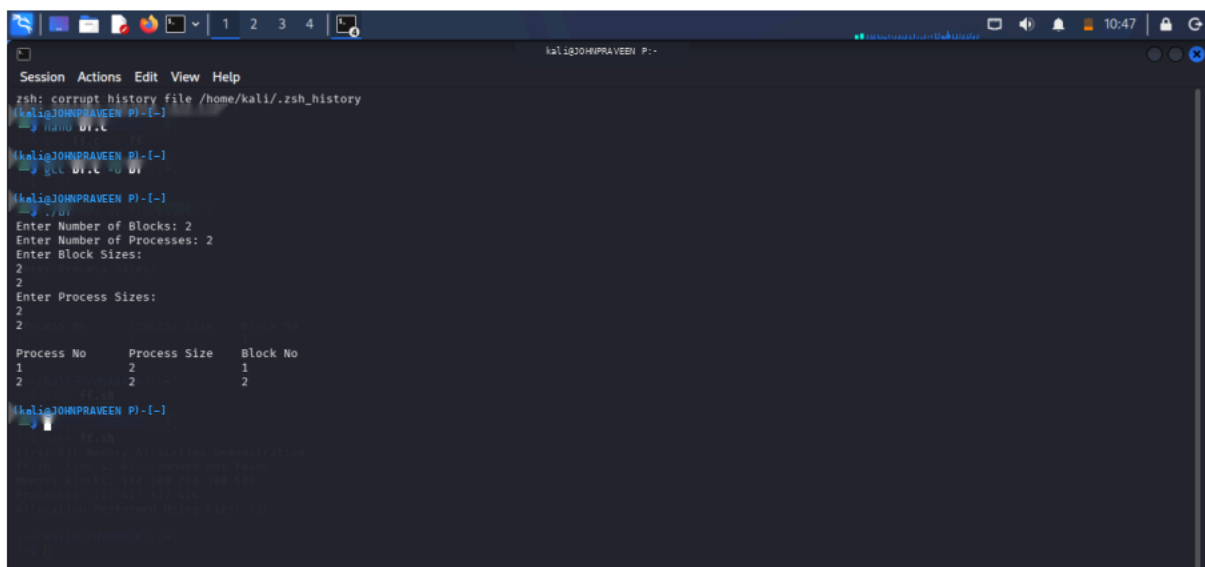
```

C PROGRAMMING : BEST FIT



```
GNU nano 9.0 bf.c
#include <stdio.h>
int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb,np,i,j,bestIdx;
    printf("Enter Number of Blocks: ");scanf("%d",&nb);
    printf("Enter Number of Processes: ");scanf("%d",&np);
    printf("Enter Block Sizes:\n");
    for(i=0;i<nb;i++)
        scanf("%d",&blockSize[i]);printf("Enter Process Sizes:\n");
    for(i=0;i<np;i++)
        scanf("%d",&processSize[i]);
    for(i=0;i<np;i++)allocation[i]=-1;
    for(i=0;i<nb;i++){bestIdx=-1;
        for(j=0;j<np;j++){
            if(blockSize[j] >= processSize[i]){
                if(bestIdx==-1 || blockSize[j] < blockSize[bestIdx])bestIdx=j;}}
        if(bestIdx!=-1){
            allocation[i]=bestIdx;
            blockSize[bestIdx]-=processSize[i];}}
    printf("\nProcess No\tProcess Size\tBlock No\n");
    for(i=0;i<np;i++){
        printf("%d\t\t%d\t\t",i+1,processSize[i]);
        if(allocation[i]!=-1)
            printf("%d\n",allocation[i]+1);
        else
            printf("Not Allocated\n");}
    return 0;}
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
kali@JOHNPAVEEN P:~$ nano bf.c
kali@JOHNPAVEEN P:~$ gcc bf.c -o bf
kali@JOHNPAVEEN P:~$ ./bf
Enter Number of Blocks: 2
Enter Number of Processes: 2
Enter Block Sizes:
2
2
Enter Process Sizes:
2
2

Process No      Process Size    Block No
1                2                1
2                2                2

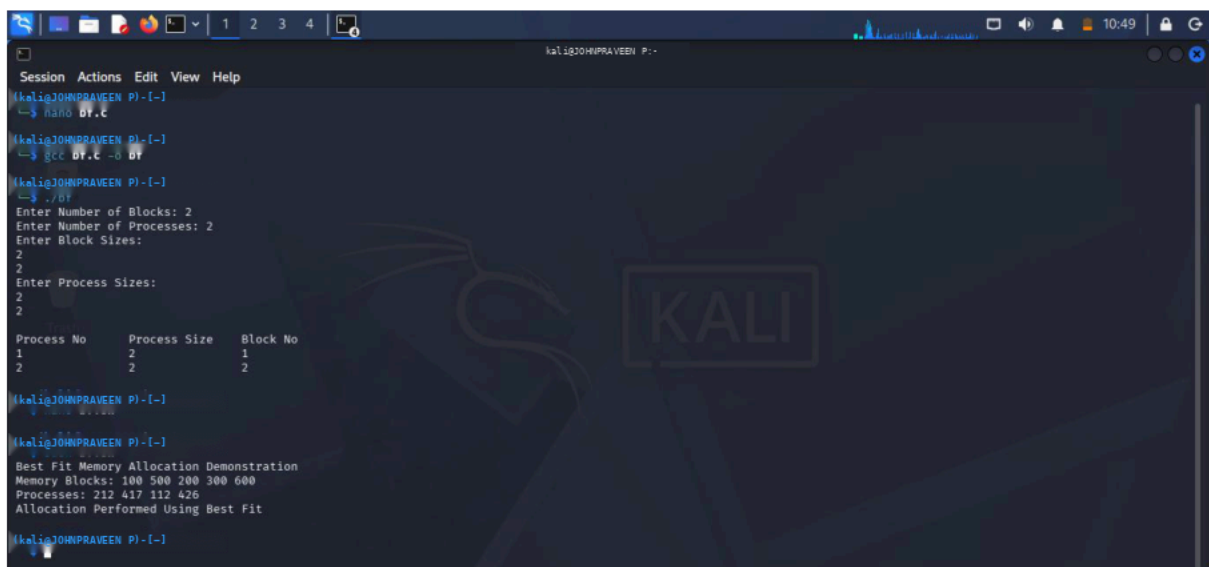
kali@JOHNPAVEEN P:~$
```

SHELL PROGRAMMING : BEST FIT



```
GNU nano 9.0 bf.sh
#!/bin/bash
echo "Best Fit Memory Allocation Demonstration"
blocks=(100 500 200 300 600)
processes=(212 417 112 426)
echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"
echo "Allocation Performed Using Best Fit"
```

OUTPUT :



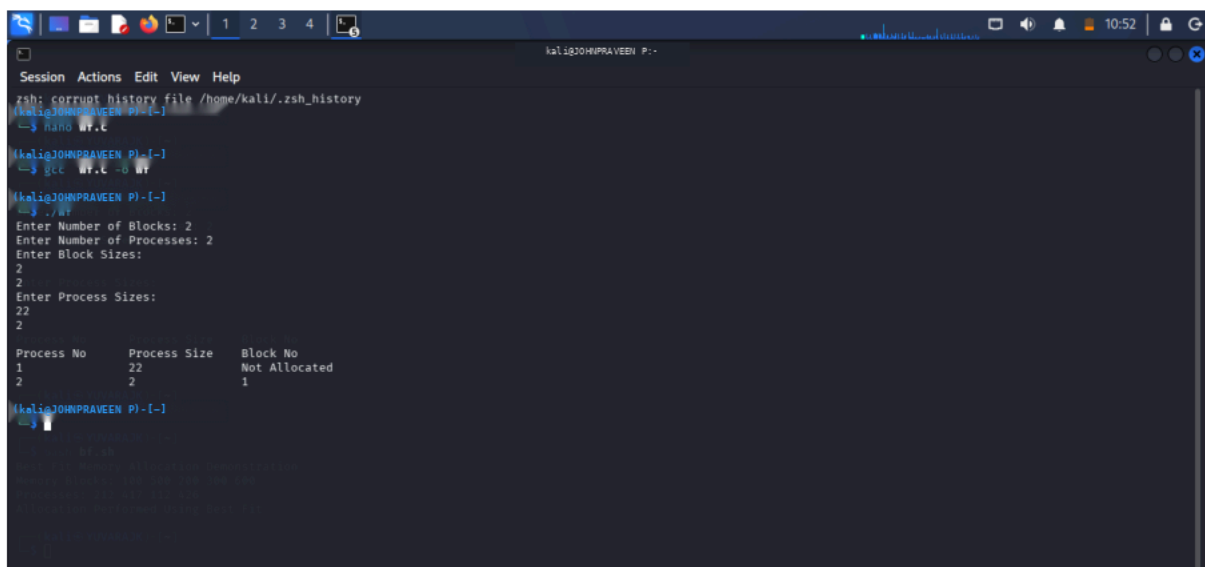
```
(kali@JOHNPRABEEN P:~)
└─$ nano bf.c
(kali@JOHNPRABEEN P:~)
└─$ gcc bf.c -o bf
(kali@JOHNPRABEEN P:~)
└─$ ./bf
Enter Number of Blocks: 2
Enter Number of Processes: 2
Enter Block Sizes:
2
2
Enter Process Sizes:
2
2
Process No      Process Size    Block No
1                2                1
2                2                2
(kali@JOHNPRABEEN P:~)
└─$ nano bf.sh
(kali@JOHNPRABEEN P:~)
└─$ ./bf.sh
Best Fit Memory Allocation Demonstration
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426
Allocation Performed Using Best Fit
(kali@JOHNPRABEEN P:~)
```

C PROGRAMMING : WORST FIT



```
GNU nano 9.0 wf.c
#include <stdio.h>
int main(){
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb,np,i,j,worstIdx;
    printf("Enter Number of Blocks: ");scanf("%d",&nb);
    printf("Enter Number of Processes: ");scanf("%d",&np);
    printf("Enter Block Sizes:\n");
    for(i=0;icnb;i++){
        scanf("%d",&blockSize[i]);printf("Enter Process Sizes:\n");
    }
    for(i=0;icnp;i++){
        scanf("%d",&processSize[i]);
    }
    for(i=0;icnp;i++){
        worstIdx=-1;
        for(j=0;jcnb;j++){
            if(blockSize[j] >= processSize[i]){
                if(worstIdx==-1 || blockSize[j] > blockSize[worstIdx])worstIdx=j;
            }
        }
        if(worstIdx!=-1){
            allocation[i]=worstIdx;
            blockSize[worstIdx]-=processSize[i];
            printf("\nProcess No\tProcess Size\tBlock No\n");
            for(i=0;icnp;i++){
                printf("%d\t\t\t\t\t",i+1,processSize[i]);
            }
            if(allocation[i]!=-1)
                printf("%d\n",allocation[i]+1);
            else
                printf("Not Allocated\n");
        }
    }
    return 0;
}
```

OUTPUT :



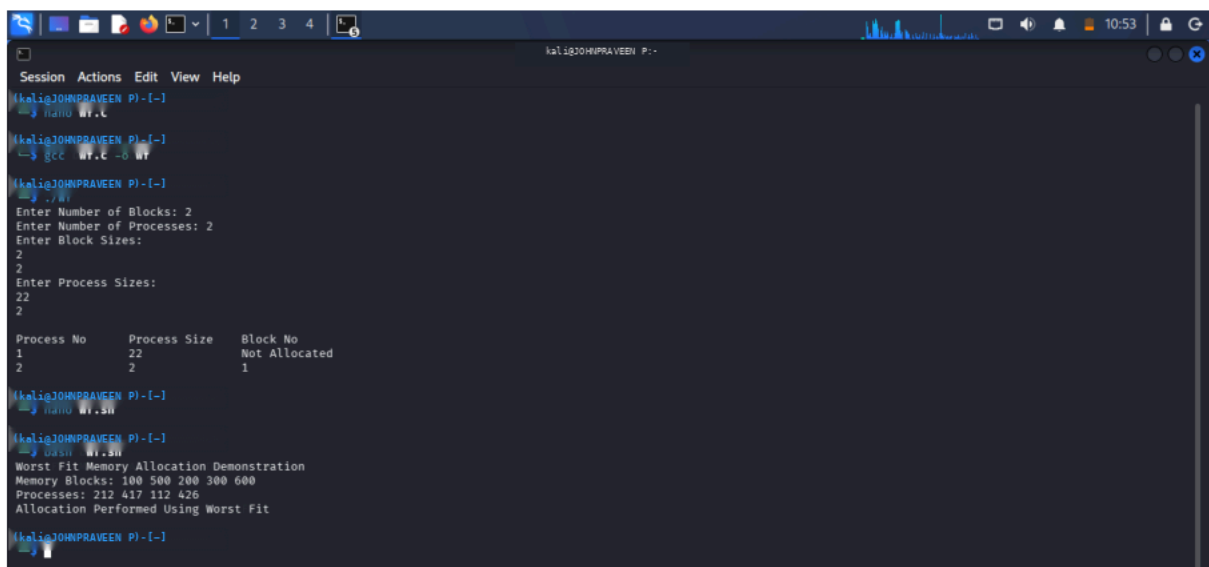
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHNPRABEEN P)-[~]
--$ nano wf.c
(kali@JOHNPRABEEN P)-[~]
--$ gcc wf.c -o wf
(kali@JOHNPRABEEN P)-[~]
--$ ./wf
Enter Number of Blocks: 2
Enter Number of Processes: 2
Enter Block Sizes:
2
2
Enter Process Sizes:
22
2
Process No      Process Size    Block No
1                22              Not Allocated
2                2               1
(kali@JOHNPRABEEN P)-[~]
--$
```

SHELL PROGRAMMING : WORST FIT



```
GNU nano 9.0 wf.sh
#!/bin/bash
echo "Worst Fit Memory Allocation Demonstration"
blocks=(100 500 200 300 600)
processes=(212 417 112 426)
echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"
echo "Allocation Performed Using Worst Fit"
```

OUTPUT :



```
(kali@JOHNPRAVEEN P:~)
$ nano wf.c
(kali@JOHNPRAVEEN P:~)
$ gcc wf.c -o wf
(kali@JOHNPRAVEEN P:~)
$ ./wf
Enter Number of Blocks: 2
Enter Number of Processes: 2
Enter Block Sizes:
2
2
Enter Process Sizes:
22
22
2
Process No      Process Size    Block No
1                22              Not Allocated
2                22              1
(kali@JOHNPRAVEEN P:~)
$ nano wf.sh
(kali@JOHNPRAVEEN P:~)
$ ./wf.sh
Worst Fit Memory Allocation Demonstration
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426
Allocation Performed Using Worst Fit
(kali@JOHNPRAVEEN P:~)
$
```